



14.3. Radix Sort

Previous Sections:

 [14.1. Bubble Sort & Comb Sort](#)

 [14.2. Selection Sort & Insertion Sort](#)

Contents:

[1. The Radix Sort Algorithm](#)

[2. Demonstration of Radix Sort](#)

[3. Implementation of Radix Sort](#)

[Summary](#)

[References](#)

In the previous sections, we learned several **comparison-based sorting algorithms**, including **Bubble Sort**, **Comb Sort**, **Selection Sort**, and **Insertion Sort**. These algorithms determine the order of elements by repeatedly comparing two values and swapping or shifting them into their correct positions.

Radix Sort takes a completely different approach. Instead of comparing elements with one another, it sorts numbers by examining each **digit** individually, beginning with the **least significant digit (units place)** and progressing toward the **most significant digit**. Because it does not rely on comparisons, Radix Sort is classified as a **non-comparison sorting algorithm**.

1. The Radix Sort Algorithm

Radix Sort processes numbers one digit at a time. During each pass, every number is placed into one of **ten buckets (0–9)** according to the current digit

being examined.

The algorithm works as follows:

1. First Pass (Units Digit)

- Place every number into one of the ten buckets based on its **units digit**.
- Collect the numbers back from Bucket 0 to Bucket 9 while preserving their order.

2. Second Pass (Tens Digit)

- Repeat the same process using the **tens digit**.

3. Continue for Higher Digits

- Continue sorting by the hundreds, thousands, and so on until every digit of the largest number has been processed.

For example, if the largest number is **2,487**, then the algorithm requires **4 passes**, since the number contains four digits.

Important Notes

- Radix Sort uses **ten buckets**, numbered from **0 to 9**.
- Each bucket behaves like a **Queue (FIFO – First In, First Out)** so that numbers with the same digit maintain their relative order.
- If some numbers contain fewer digits than others, leading zeros are assumed.
 - For example, $5 \rightarrow 005$; $32 \rightarrow 032$; $248 \rightarrow 248$
 - This ensures that every number has the same number of digits during processing.
- Time Complexity

If **n** = number of elements; **d** = number of digits of the largest element. Then the time complexity is $O(n \times d)$.

Radix Sort performs especially well when sorting a large collection of integers that contain many digits.

2. Demonstration of Radix Sort

Consider the following sequence:

Initial Array

```
+-----+-----+-----+-----+-----+-----+-----+
| 432 | 121 | 223 | 145 | 564 | 788 | 101 |
+-----+-----+-----+-----+-----+-----+-----+
```

The largest number has **3 digits**, so the algorithm performs **three passes**: Pass 1 – Units digit; Pass 2 – Tens digit; Pass 3 – Hundreds digit

- Pass 1 – Sort by Units Digit

Bucket 0	Bucket 1	Bucket 2	Bucket 3
	101	432	223
	121		
__	__	__	__
Bucket 4	Bucket 5	Bucket 6	Bucket 7
564	145		
__	__	__	__
Bucket 8	Bucket 9		
788			
__	__		

This means: **121** entered the queue first. **101** entered second. When the bucket is emptied (FIFO): 121 leaves first; 101 leaves second.

So the collected sequence becomes:

```
+-----+-----+-----+-----+-----+-----+-----+
| 121 | 101 | 432 | 223 | 564 | 145 | 788 |
+-----+-----+-----+-----+-----+-----+-----+
```

- Pass 2 – Sort by Tens Digit

Current array:

```
+-----+-----+-----+-----+-----+-----+-----+
| 121 | 101 | 432 | 223 | 564 | 145 | 788 |
+-----+-----+-----+-----+-----+-----+-----+
```

Bucket 0

```
|101|
|  |
|__|
```

Bucket 1

```
|  |
|  |
|__|
```

Bucket 2

```
|223|
|121|
|__|
```

Bucket 3

```
|432|
|  |
|__|
```

Bucket 4

```
|145|
|  |
|__|
```

Bucket 5

```
|  |
|  |
|__|
```

Bucket 6

```
|564|
|  |
|__|
```

Bucket 7

```
|  |
|  |
|__|
```

Bucket 8

```
|788|
|  |
|__|
```

Bucket 9

```
|  |
|  |
|__|
```

Collecting the buckets produces

```
+-----+-----+-----+-----+-----+-----+-----+
| 101 | 121 | 223 | 432 | 145 | 564 | 788 |
+-----+-----+-----+-----+-----+-----+-----+
```

- Pass 3 – Sort by Hundreds Digit

Current array:

```
+-----+-----+-----+-----+-----+-----+-----+
| 101 | 121 | 223 | 432 | 145 | 564 | 788 |
+-----+-----+-----+-----+-----+-----+-----+
```

Hundreds digits:

Bucket 0	Bucket 1	Bucket 2	Bucket 3
 __	145 121 101	223 __	 __
Bucket 4	Bucket 5	Bucket 6	Bucket 7
432 __	564 __	 __	788 __
Bucket 8	Bucket 9		
 __	 __		

Collecting the buckets one final time gives:

```
+-----+-----+-----+-----+-----+-----+
| 101 | 121 | 145 | 223 | 432 | 564 | 788 |
+-----+-----+-----+-----+-----+-----+
```

Which is also the final sorted array

3. Implementation of Radix Sort

Below is the implementation of Radix Sort using Python

```
def radix_sort(a): # [Time(n), Space(n) = O(n * d), O(n *
    d)]
    max_length = False
    temp, placement = -1, 1
    while not max_length:
        max_length = True
        # Create a bucket to store the numbers from index 0
        to index 9
        buckets = [list() for _ in range(10)] # Multi-dime
```

```

nsional List
    for i in a:
        temp = i // placement # Take the number as temp
        # Find the digit of the number from last to first
        digit = temp % 10
        # Append the number to the array in the right index (based on digit)
        buckets[digit].append(i)
        # If the length is maximum and no more temp
        if max_length and temp > 0:
            max_length = False
    # Initial Index
    v = 0
    for b in range(10):
        # Iterate through the bucket
        bucket = buckets[b]
        for i in bucket:
            a[v] = i # Take the number in the bucket
            v += 1
    # Proceed to the next digit
    placement *= 10

    return a

```

Summary

Unlike Bubble Sort, Selection Sort, or Insertion Sort, **Radix Sort never compares two elements directly.**

- Instead, it repeatedly distributes numbers into **ten queues (buckets)** according to one digit at a time, beginning with the least significant digit and ending with the most significant digit.
- After processing every digit, the numbers become fully sorted.

Because each bucket preserves insertion order using the **FIFO principle**, Radix Sort is a **stable sorting algorithm**. Its running time is $O(n \times d)$, making it

particularly efficient when sorting large collections of integers with a fixed number of digits.

References

https://drive.google.com/file/d/1jK22rfWszKKgPgeprquGoeJKqgBg8rUY/view?usp=drive_link

<https://www.geeksforgeeks.org/dsa/radix-sort/>

<https://www.programiz.com/dsa/radix-sort>

Next Section:

 [14.4. Counting Sort](#)